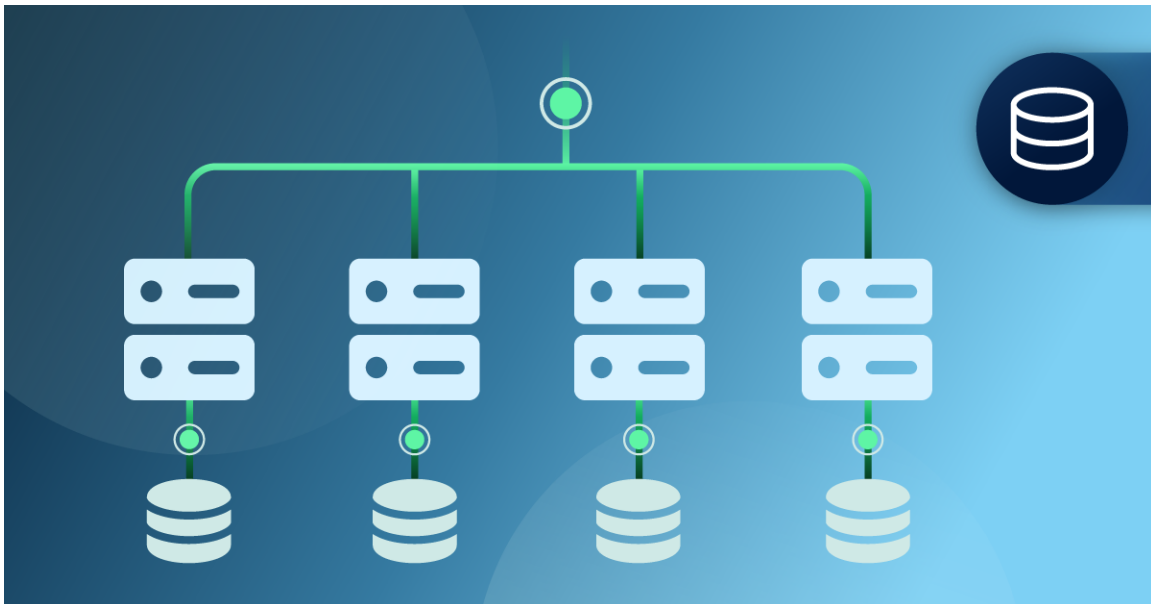


Parallel Log Processing & Analytics Engine

OS Project Proposal



Members:

Shivam (24k-0957)

Yogita (24K-0929)

Jeevan (24K-0821)

DESCRIPTION

HOW IT WILL WORK:

This project aims to build a high-performance engine capable of processing massive log files efficiently. Instead of reading logs sequentially—which is slow and inefficient for large datasets—the engine will divide large log files into smaller segments. These segments will be assigned to a pool of worker threads that will process the data concurrently.

As each thread parses its assigned chunk of logs, it will extract relevant data (such as errors or specific event patterns) and update shared global statistics. To prevent race conditions and ensure our final data is accurate, we will use **mutexes** to protect the global counters during updates. Additionally, we will implement **semaphores** to strictly control disk access, ensuring that multiple threads trying to read from the disk simultaneously do not cause I/O bottlenecks. Ultimately, this system will mimic real-world security monitoring tools by providing real-time analytics, teaching us the practicalities of parallel file processing and synchronization overhead.

FUNCTIONAL FEATURES:

1. **Log Segmentation:** Automatically divides large, monolithic log files into smaller, manageable chunks for distribution across threads.

2. **Concurrent Parsing Engine:** Spawns and manages multiple threads to simultaneously read and parse different log segments.
3. **I/O Access Control:** Uses semaphores to regulate thread access to the disk, preventing resource starvation.
4. **Severity Classification:** Analyzes log lines and categorizes them by severity level (e.g., INFO, WARNING, ERROR, CRITICAL) on the fly.
5. **Pattern & Anomaly Detection:** Scans the text within log segments to identify specific recurring keywords or security flags.
6. **Thread-Safe Aggregation:** Utilizes mutexes to safely lock and update a central statistics registry as threads finish their processing.
7. **Speed Benchmarking:** Calculates and compares the execution time of processing the logs sequentially versus processing them in parallel.
8. **Report Export:** Compiles the aggregated data and outputs a structured summary report file at the end of the execution.

PLAN OF WORK

Short-Term Goals:

- Set up the shared development environment and version control (e.g., GitHub).
- Write a script to generate massive dummy log files for testing purposes.
- Implement the basic file I/O operations (reading files into memory).
- Research and set up the basic thread pool structure using our chosen programming language.

Division of Contributions:

- **Shivam will contribute:**
 - **Log Segmentation:** Implementing the logic to correctly split files without cutting log lines in half.
 - **Concurrent Parsing Engine:** Writing the core threading logic and assigning tasks to workers.
 - **I/O Access Control:** Implementing the semaphores to manage how threads read the file chunks.
- **Yogita will contribute:**
 - **Pattern & Anomaly Detection:** Implementing the search algorithms to flag specific text patterns.
 - **Severity Classification:** Writing the string-matching logic to categorize the incoming log lines.

- **Speed Benchmarking:** Building the timer functions to track and compare sequential vs. parallel performance.
- **Jeevan will contribute:**
 - **Thread-Safe Aggregation:** Building the shared global counters and implementing the mutex locks to protect them.
 - **Report Export:** Creating the final output generator that formats the aggregated data into a usable file.

REFERENCE

☐ Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley. *(Used for concepts on mutexes, semaphores, and concurrency)*

☐ [Insert official documentation for the language you are using, e.g., POSIX Threads (pthreads) API Documentation / Java Concurrency API / C++ std::thread reference]

☐ [Insert any specific article or tutorial you might use for log parsing algorithms]